

1. Contexto: El Estado Actual del Código sin Patrones

Para que los patrones tengan sentido, el estudiante debe primero ver el dolor que produce su ausencia. La siguiente tabla describe el estado del código BovWeight CR antes de aplicar cada patrón:

Patrón	Problema actual (sin patrón)	Síntoma observable	Consecuencia a largo plazo
Factory	new Brahman() y new Nelore() dispersos en 6 controladores distintos	Cambiar el constructor de Raza rompe 6 archivos al mismo tiempo	Imposible añadir nueva raza sin buscar todos los puntos de creación
Repository	Animal::where() repetido en ReporteService, EstimadorService, DashboardController	Cambiar el ORM (de Eloquent a Doctrine) obliga a modificar toda la capa de negocio	Lógica de negocio acoplada a detalles de persistencia; imposible pruebas unitarias
Observer	Después de guardar un RegistroPeso el controlador llama: notificar, actualizar dashboard, recalcular ICC, disparar webhook	Un nuevo suscriptor requiere tocar el método central	El método crece a 80+ líneas; acoplamiento entre registro de peso y sus efectos secundarios
Strategy	if (\$método === 'yolov8') {...} elseif (\$método === 'regresion') {...} elseif (\$método === 'tabla') {...}	Agregar nuevo algoritmo requiere abrir EstimadorPesoService	Lógica de comparación mezclada con orquestación del flujo

2. Instrucciones por Patron

Para cada patrón siga el mismo proceso de cuatro pasos en el orden indicado. No salte al código antes de completar el análisis.

- Lea la tarjeta de contexto del patrón.
- Implemente el patrón en código PHP.

PATRÓN FACTORY Factory Method [Creacional - GoF: Factory Method]

Problema que resuelve: La creación de objetos está dispersa por el código cliente. Cuando cambia el constructor o se añade un nuevo tipo, hay que modificar múltiples puntos de creación.

Solución general: Define un método o clase cuya única responsabilidad es crear instancias del tipo correcto según el contexto. El código cliente depende de la abstracción, no del constructor concreto.

Cuando usarlo: Cuando existe más de un tipo concreto de un mismo concepto (Brahman, Nelore, Angus) y la lógica de cual instanciar debe estar centralizada.

Patrón Factory - Caso BovWeight CR: RazaFactory

El problema concreto: en el proyecto existen al menos 6 archivos que crean instancias de Brahman o Nelore directamente con new. Si mañana SENASA agrega la raza Angus al programa de mejoramiento genético, se deben encontrar y modificar esos 6 puntos.

Participantes GoF en este contexto

Creator (interfaz): IRazaFactory - define el método create(string \$nombreRaza): Raza

ConcreteCreator: RazaFactory - implementa el método, decide qué subclase instanciar según el nombre de raza

Product (abstracción): Raza - clase abstracta del Lab 3-A

ConcreteProduct: Brahman, Nelore (y futuro Angus sin modificar factory)

Instrucciones de implementación - Factory

- Cree la interfaz IRazaFactory con un método create(string \$nombreRaza): Raza.
- Implemente RazaFactory que mapee el string al constructor correcto usando un array asociativo, no un switch.
- Agréguela al Service Container de Laravel como singleton.
- Refactorice al menos 2 puntos de creación en el código existente para que usen la factory vía inyección de dependencias.

PATRÓN REPO Repository [Estructural - GoF: No GoF clásico - DDD / Fowler]

Problema que resuelve: La lógica de consulta a base de datos está mezclada con la lógica de negocio. Cambiar el ORM o la fuente de datos afecta directamente los servicios.

Solución general: Abstrae la capa de persistencia detrás de una interfaz que habla el lenguaje del dominio (getByArete, getByRancho) en lugar del lenguaje SQL/ORM.

Cuando usarlo: Cuando hay lógica de negocio que necesita datos pero no debería saber de dónde vienen. Especialmente útil para hacer pruebas unitarias con datos en memoria.

Patrón Repository - Caso BovWeight CR: AnimalRepository

El problema concreto: `Animal::where('rancho_id', $id)->with('registrosPeso')->get()` aparece en `ReporteService`, `EstimadorService` y `DashboardController`. Cuando el equipo decide agregar cache Redis para esta consulta, hay que encontrar los 3 puntos y modificarlos individualmente.

Participantes en este contexto

Repository Interface: `IAntimalRepository` - métodos con nombres del dominio, sin rastro de Eloquent

Concrete Repository: `EloquentAnimalRepository` - implementa con Eloquent (intercambiable por `DoctrineAnimalRepository`)

In-Memory Repository (para tests): `InMemoryAnimalRepository` - implementa con un array, sin tocar la BD

Instrucciones de implementación - Repository

- Defina la interfaz `IAntimalRepository` con al menos los métodos: `findByArete(string): ?Animal`, `findAllByRancho(int): array`, `save(Animal): void`, `delete(int): void`.
- Implemente `EloquentAnimalRepository` que traduzca cada método al ORM correspondiente.
- Implemente `InMemoryAnimalRepository` con un array privado para uso en pruebas, sin base de datos.
- Registre en el `ServiceProvider`: `$this->app->bind(IAntimalRepository::class, EloquentAnimalRepository::class)`.
- Refactorice `ReporteService` para que reciba `IAntimalRepository` por constructor injection en lugar de llamar `Animal::` directamente.

PATRÓN OBSERVER Observer [Comportamiento - GoF: Observer (GoF)]

Problema que resuelve: Un evento central (registro de peso) debe notificar a múltiples subsistemas. El emisor llama directamente a cada receptor, creando acoplamiento fuerte.

Solución general: Define una relación uno-a-muchos entre objetos donde el sujeto notifica a todos sus observadores automáticamente. El sujeto no conoce quienes son sus observadores.

Cuando usarlo: Cuando un evento en un objeto debe desencadenar reacciones en múltiples objetos sin que el emisor conozca los receptores ni la cantidad de ellos.

Patrón Observer - Caso BovWeight CR: Eventos de RegistroPeso

El problema concreto: cuando se registra un nuevo peso, el controlador ejecuta en secuencia: envía email al propietario, actualiza el dashboard, recalcula el ICC, dispara un webhook a SENASA. Si el equipo de operaciones quiere agregar alertas SMS, deben abrir el controlador y agregar la llamada.

Participantes GoF en este contexto

Subject (Observable): RegistroPesoSubject - mantiene lista de observadores, emite notificaciones al guardar un peso

Observer (interfaz): IRegistroPesoObserver - define onPesoRegistrado(RegistroPeso \$registro): void

ConcreteObservers: NotificadorPropietario, ActualizadorDashboard, RecalculadorICC, WebhookSenasa

Nota Laravel: En Laravel esto puede implementarse con Events/Listeners, que son la implementación del patrón Observer de Eloquent.

Instrucciones de implementación - Observer

- Defina la interfaz IRegistroPesoObserver con el método onPesoRegistrado(RegistroPeso \$registro): void.
- Cree RegistroPesoSubject con métodos: suscribir(IRegistroPesoObserver): void, desuscribir(IRegistroPesoObserver): void, y notificar(RegistroPeso): void privado.
- Implemente al menos 3 observadores concretos: NotificadorPropietario, RecalculadorICC, WebhookSenasa.
- Demuestre que agregar un cuarto observador (AlertaSMS) no requiere modificar RegistroPesoSubject ni ningún observador existente.
- Escriba 1 prueba que verifique que al llamar notificar(), todos los observadores suscritos reciben la llamada (use mocks o spies).

PATRÓN STRATEGY Strategy *[Comportamiento - GoF: Strategy (GoF)]*

Problema que resuelve: Un algoritmo tiene múltiples variantes y la selección ocurre mediante if-else/switch. Agregar una nueva variante requiere modificar la clase que lo orquesta.

Solución general: Encapsula cada variante del algoritmo en su propia clase que implementa una interfaz común. El contexto delega la ejecución a la estrategia inyectada, sin conocer cuál es.

Cuando usarlo: Cuando existen múltiples algoritmos intercambiables para resolver el mismo problema, y la elección puede variar en tiempo de ejecución o por configuración.

Patrón Strategy - Caso BovWeight CR: Algoritmos de Estimación de Peso

El problema concreto: BovWeight CR soporta tres métodos de estimación de peso: YOLOv8, Regresión Lineal y Tabla de Referencia. Actualmente estos tres algoritmos viven como bloques if-else en EstimadorPesoService, junto con la lógica de orquestación del flujo.

Participantes GoF en este contexto

Strategy (interfaz): IAlgoritmoEstimacion - define ejecutar(array \$datos): ResultadoEstimacion

ConcreteStrategies: AlgoritmoYolov8, AlgoritmoRegresionLineal, AlgoritmoTablaReferencia

Context: EstimadorPesoService - recibe IAlgoritmoEstimacion por constructor, lo llama sin conocer la implementación

Value Object resultado: ResultadoEstimacion - encapsula peso_kg, confianza, metodo_usado (inmutable, sin setters)

Instrucciones de implementación - Strategy

- Defina la interfaz IAlgoritmoEstimacion con el método ejecutar(array \$datosEntrada): ResultadoEstimacion.
- Cree el value object ResultadoEstimacion con propiedades readonly: pesoKg (float), confianzaPorcentaje (float), metodoUsado (string). Sin setters.
- Implemente las tres estrategias concretas. AlgoritmoYolov8 puede simular la llamada HTTP. AlgoritmoRegresionLineal y AlgoritmoTablaReferencia pueden usar formulas simplificadas.
- Refactorice EstimadorPesoService para que reciba IAlgoritmoEstimacion por constructor injection. El método estimar() debe eliminar completamente el if-else.
- Demuestre cambio de estrategia en tiempo de ejecución: use la tabla de referencia cuando no hay conexión al servicio YOLOv8 (fallback).